C1 Fabric architecture

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C1 Fabric architecture 3
C1-L1 Tenant, capacity, workspace and item 3
C1-L2 OneLake, one lake per tenant 7
C1-L3 Workspace roles, granted to groups 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C1 Fabric architecture

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Workspaces, capacities, items and OneLake as one logical lake.

C1-L1 Tenant, capacity, workspace and item

Concept

Everything in Fabric sits in one hierarchy. Learn it first, because cost, access and deployment all follow it.

- **Tenant.** Your organisation's Microsoft Entra ID tenant. Tenant settings, set by Fabric administrators, apply to everything below.
- **Capacity.** A pool of compute you buy. Every Fabric workload, from Spark to Power BI, draws from the capacity its workspace is assigned to.
- **Workspace.** A container where a team creates and shares items. Access is granted per workspace, through roles (lesson C1-L3).
- **Item.** A lakehouse, warehouse, notebook, pipeline, semantic model, report, eventhouse and so on. Items hold or process the data.

Capacities come as SKUs. F SKUs are the recommended Fabric capacities. The number is the compute in **capacity units** (CUs): an F2 has 2 CUs, an F64 has 64, up to F2048. A trial capacity has 64 CUs. Microsoft 365 P SKUs also run Fabric if Fabric is enabled on them.

Size affects licensing, not only speed. On **F64 or larger** (or a P capacity), users with a free Fabric license can view Power BI content. On smaller F SKUs, each viewer needs a Pro, Premium Per User or individual trial licence.

Pausing a capacity stops compute billing while it is paused. It is not free of consequences: when you pause, any smoothed and overage usage still owed is added to your Azure bill at once, and items on that capacity stop working until you resume.

Two practical rules follow from the hierarchy:

- Put dev, test and production in **separate workspaces**, and assign production to a capacity that experiments cannot exhaust.
- Before you design anything, find out which capacity SKU you have. It decides what users need to pay for and how much work runs at once.

Facts in this lesson come from Microsoft Learn's licensing and capacity pages, checked September 2026.

Why it matters

A team built a Power BI report for 300 frontline managers on an F32 capacity, assuming that managers with free licences could view it, as they had on the old Premium P1. Launch day produced 300 "You need a license" messages. The fix was either 300 Pro licences or a move to F64. Neither was in the budget, and the launch slipped a quarter.

Worked example: weak code

```
Workspace: Analytics (all projects)
Capacity: F32 shared with the data science sandbox
Viewers: 300 managers with free licences
Environments: dev and prod items in the same workspace
```

1. **Line 2.** Production shares compute with experiments. One heavy notebook throttles the reports.
2. **Line 3.** Below F64, every viewer needs a Pro, PPU or trial licence.
3. **Line 4.** No separation: a change to a dev item can break production.

Worked example: strong code

```
Workspace: sales_reporting_prod   -> capacity: F64 (production only)
Workspace: sales_reporting_test   -> capacity: F8
Workspace: sales_reporting_dev    -> capacity: F8
Viewers: 300 managers with free licences, reading the prod workspace's app
Sandbox workspaces                -> capacity: F4, paused out of hours
```

1. **Line 1.** F64 lets free-licence users view Power BI content, and production has its own compute.
2. **Line 5.** Experiments cannot starve production, and pausing saves money when nobody works.

The capacity decides who can view, how much runs at once and what it costs. Separate environments into workspaces, and give production its own capacity.

Recap card

- **Fabric hierarchy** Tenant, capacity, workspace, item.
- **What F64 means** An F SKU with 64 capacity units. F SKUs run from F2 to F2048.
- **Free-licence viewers of Power BI content** Need an F64 or larger (or P) capacity. Smaller F SKUs need Pro, PPU or trial licences for viewers.
- **Pausing a capacity** Stops compute billing from then on, bills remaining smoothed and overage usage at once, and stops its items.

Practice

Exercise 1. Multiple choice (level 1)

What does the number in the SKU name F32 tell you?

- a) 32 users can use the capacity.
- b) The capacity has 32 capacity units (CUs) of compute.
- c) 32 GB of OneLake storage is included.
- d) 32 workspaces can be assigned to it.

Why?

- r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.
- r2) SKU numbers are marketing names with no meaning.

Exercise 2. Put in order (level 2)

Order the Fabric hierarchy from the broadest to the most specific.

- Workspace
- Capacity
- Tenant
- Item, such as a lakehouse

Exercise 3. Architecture design (level 2)

A report will serve 200 viewers who have only free Fabric licences. Choose the set-up.

Options: F64 or larger, F16, cheaper, same features.

1. Capacity for the production workspace
2. What else runs on that capacity

Exercise 4. Write the function (level 3)

Write `cu_hours(sku: str, hours: float) -> float` for a cost estimate. The SKU is a string such as `"F64"`. Return the capacity units multiplied by the hours running. Accept only the F SKUs F2, F4, F8, F16, F32, F64, F128, F256, F512, F1024 and F2048, and raise `ValueError` for any other SKU or for negative hours.

Your code must pass these checks:

- Multiply the SKU's CUs by hours: F64 for 24 hours is 1,536 CU hours.

- Only the listed F SKUs are valid. F3, P1, lower-case f64, F4096 and 64 must raise ValueError.
- Negative hours must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
F_SKUS = ...

# Capacity-unit hours for running an F SKU capacity.
def cu_hours(sku: str, hours: float) -> float: ...
```

Starter code:

```python
def cu_hours(sku, hours):
    return int(sku[1:]) * hours
```

Exercise 5. Multiple choice (level 4)

A team pauses its F64 capacity at 18:00 to save money after a heavy afternoon of Spark jobs. What happens?

- a) Nothing is billed for the afternoon's work, because the capacity is paused.
- b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes.
- c) The capacity keeps running reports but stops Spark.
- d) OneLake data on the capacity is deleted after 24 hours.

Why?

- r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.
- r2) Paused capacities are billed at double rate.

Sources

- F SKUs from F2 to F2048, where the number equals capacity units; a trial capacity has 64 CUs. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/licenses (updated 2026-06-15))
- F64+ or P capacities support viewing Power BI content by users with a free Fabric licence; smaller F SKUs require Pro, PPU or trial licences for viewers. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/licenses (updated 2026-06-15))
- When a capacity is paused, remaining cumulative overages and smoothed operations are added to the Azure bill. (verified: https://

learn.microsoft.com/en-us/fabric/enterprise/pause-resume (updated 2026-04-20): pausing can prevent content from being available until resumed)

C1-L2 OneLake, one lake per tenant

Concept

Every Fabric tenant has exactly **one OneLake**. You cannot create a second one or delete it. It is built on Azure Data Lake Storage and stores tables in open formats: **Delta Parquet** or **Iceberg**. Every Fabric engine (Spark, the SQL engines, Power BI's Direct Lake, KQL) reads the same files, so one copy of the data serves them all.

OneLake mirrors the Fabric hierarchy: a folder per **workspace**, and inside it a folder per **item**, named with its type. A lakehouse holds two areas: `Tables` for managed Delta tables and `Files` for anything else.

You address data with an ADLS-style path:

```
1 https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.<itemtype>/
    <path>
2 abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/
    <path>
```

For example `abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/Tables/orders`. The item type is lower case: `.lakehouse`, `.warehouse`.

Names change. **GUIDs do not.** OneLake accepts the workspace and item GUIDs in place of names, and they stay fixed when someone renames the workspace or item. With GUIDs you give both the workspace and the item as GUIDs, and you drop the item type. You find them in the item's URL in the Fabric portal.

Use GUIDs in pipelines, jobs and code that other people will run. Two more reasons from Microsoft Learn:

- The `abfs` driver URI does not allow special characters such as spaces in the workspace name. GUIDs avoid the problem.
- The global endpoint `onelake.dfs.fabric.microsoft.com` can route through another region. If data residency matters, use your workspace's regional endpoint, `https://<region>-onelake.dfs.fabric.microsoft.com`.

Because tools that speak ADLS APIs can read OneLake, Azure Databricks, Azure Storage Explorer or Python's Azure SDK can read Fabric data with the right permissions.

Why it matters

A Databricks job read a Fabric lakehouse through `abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/...`. It failed on the first run, because the workspace name held a space. The engineer renamed the workspace to `SalesAnalytics`. The job worked, and four Power BI dataflows and two pipelines that referred to the old name broke that night. Using the workspace and item GUIDs would have avoided both failures.

Worked example: weak code

```
1 path = "abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/Sales LH.Lakehouse/Tables/orders"
2 df = spark.read.format("delta").load(path)
```

1. **Line 1.** Spaces break the abfs URI, the item type is not lower case, and a rename breaks the path.

Worked example: strong code

```
1  ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
2
3
4  def onelake_table_path(workspace_id: str, item_id: str, table: str) -> str:
5      """abfss path to a lakehouse table, by GUID, stable across renames."""
1      return f"abfss://{workspace_id}@{ONELAKE_HOST}/{item_id}/Tables/{table}"
7
8
2  path = onelake_table_path(CONFIG["workspace_id"], CONFIG["lakehouse_id"], "orders")
10 df = spark.read.format("delta").load(path)
```

1. **Line 6.** With GUIDs, both workspace and item are IDs and the item type is dropped.
2. **Line 9.** IDs come from config, not from code (lesson A1-L3).

One lake, one copy, many engines. Address data by GUID in anything automated.

Recap card

- **How many OneLakes per tenant?** Exactly one. You cannot create or delete OneLakes.

- **OneLake table formats** Delta Parquet or Iceberg, open formats every Fabric engine reads.
- **OneLake path by name** abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/Tables/<table>.
- **Why GUIDs in automation** They survive renames, avoid special-character problems in abfs URIs, and drop the item type.

Practice

Exercise 1. Multiple choice (level 1)

Your company has three business units in one Fabric tenant. How many OneLakes are there?

- a) One per business unit
- b) One for the whole tenant
- c) One per capacity
- d) One per workspace

Why?

- r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.
- r2) OneLake licences are sold per tenant.

Exercise 2. Fill the gap (level 2)

Complete the path to table `orders` in lakehouse `lh_sales`, workspace `sales_prod`.

```
abfss://_____(1)@onelake.dfs.fabric.microsoft.com/lh_sales._____(2)/
    _____(3)/orders
```

Exercise 3. Spot the bug (level 3)

This path fails with an invalid URI error. Tap the line with the problem and rewrite the path so it survives renames too.

```
WORKSPACE = "Sales Analytics"
LAKEHOUSE = "lh_sales"
path = f"abfss://{WORKSPACE}@onelake.dfs.fabric.microsoft.com/
    {LAKEHOUSE}.lakehouse/Tables/orders"
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `onelake_path(workspace: str, item: str, item_type: str, relative: str) -> str` returning `abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<item_type>/<relative>`.

- `item_type` must be `"lakehouse"` or `"warehouse"`. Otherwise raise `ValueError`.
- Raise `ValueError` if `workspace` or `item` contains a space.
- Strip leading slashes from `relative`.

Your code must pass these checks:

- Build the path in the given format, without a double slash.
- Only lower-case lakehouse or warehouse are allowed.
- Names with spaces break the abfs URI. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ONELAKE_HOST = ...

# Module constant.
ITEM_TYPES = ...

# abfss path to data in a OneLake item, addressed by name.
def onelake_path(
    workspace: str,
    item: str,
    item_type: str,
    relative: str,
) -> str: ...
```

Starter code:

```
def onelake_path(workspace, item, item_type, relative):
    return f"abfss://{workspace}@onelake.dfs.fabric.microsoft.com/{item}.{item_type}/{relative}"
```

Exercise 5. Architecture design (level 4)

An Azure Databricks job in the UK will read a Fabric lakehouse in a UK South workspace every night. Data must stay in the UK. Choose the set-up.

Options: GUIDs from config, Display names in the code.

1. How the job refers to workspace and lakehouse
2. Endpoint
3. Data movement

Sources

- Every tenant has a single OneLake, built on ADLS, storing tables in Delta Parquet or Iceberg; you cannot create multiple OneLakes. (verified: https://learn.microsoft.com/en-us/fabric/onelake/onelake-overview (updated 2026-03-09))
- OneLake URI formats by name and by GUID; GUIDs do not change on rename; abfs URIs do not allow special characters such as spaces in the workspace name; the global endpoint may route data out of region, and regional endpoints avoid this. (verified: https://learn.microsoft.com/en-us/fabric/onelake/onelake-access-api (updated 2026-05-18))

C1-L3 Workspace roles, granted to groups

Concept

A workspace has four roles: **Admin**, **Member**, **Contributor** and **Viewer**. Each role includes the rights of the ones below it, with a few rights reserved for the top.

From Microsoft Learn's role table (checked September 2026):

Right	Admin	Member	Contributor	Viewer
Update or delete the workspace, add or remove admins	yes			
Connect the workspace to Git, create a workspace identity	yes			
Add members or lower roles, allow resharing	yes	yes		
Create and change items, run notebooks and pipelines	yes	yes	yes	
Read lakehouse and warehouse data through Spark and OneLake APIs (ReadAll)	yes	yes	yes	
Read lakehouse and warehouse data with T-SQL (ReadData)	yes	yes	yes	yes

Two consequences surprise people coming from Databricks:

- A **Viewer** can query a lakehouse through its SQL analytics endpoint, but cannot read the same tables from a Spark notebook or through OneLake APIs.

- A **Contributor** can change and delete every item in the workspace, including production lakehouse tables. Contributor in production is powerful.

Rules that keep access manageable:

- **Assign roles to Microsoft Entra security groups**, not to individuals (as in lesson A4-L2). People join and leave groups. Role assignments stay the same.
- **Least privilege per environment.** Engineers are Contributors in dev. In production, deployment runs under an automation identity, and people are Viewers unless they are on call.
- **Keep at least two admins**, as groups or named people, so a departure does not lock the workspace.
- Workspace roles are coarse. For finer control inside an item, use item sharing, SQL permissions, or OneLake security (lesson C9-L2).

Why it matters

A contractor was added to a production workspace as Contributor "to look at a notebook". Two weeks later they ran a clean-up notebook against the wrong lakehouse and dropped three tables that reports depended on. Viewer would have let them read the notebook and query the data with T-SQL, and would not have let them change anything.

Worked example: weak code

```
1 workspace: sales_prod
2 roles:
1   Admin: [sam@contoso.com]
2   Contributor: [grp-sales-analysts, contractor@partner.com]
5   Viewer: []
```

1. **Line 3.** One named admin: if Sam leaves, nobody can manage the workspace.
2. **Line 4.** Write access to production for people who need to read, including an external individual.

Worked example: strong code

```
1 workspace: sales_prod
2 roles:
1   Admin: [grp-fabric-platform-admins]
4   Member: [grp-sales-data-leads]
2   Contributor: [sp-sales-deployment]
3   Viewer: [grp-sales-analysts, grp-sales-contractors]
```

1. **Line 3.** A group of admins: no single point of failure.

2. **Line 5.** Only the deployment identity writes to production.
3. **Line 6.** Readers query with T-SQL and reports, and cannot change items.

Roles go to groups, write access in production goes to automation, and people read.

Recap card

- **Fabric workspace roles** Admin, Member, Contributor, Viewer.
- **Admin-only rights** Update or delete the workspace, manage admins, connect Git, create a workspace identity.
- **Viewer and lakehouse data** Reads with T-SQL (ReadData). Not through Spark or OneLake APIs (ReadAll starts at Contributor).
- **Who writes to production?** An automation identity as Contributor. People get Viewer, through groups.

Practice

Exercise 1. Multiple choice (level 1)

Which workspace role can connect a workspace to a Git repository?

- a) Admin only
- b) Admin and Member
- c) Any role except Viewer
- d) Any role

Why?

- r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.
- r2) Git is a paid feature for Admins.

Exercise 2. Multiple choice (level 2)

An analyst with the Viewer role runs `spark.read.table("lh_sales.orders")` in a notebook in that workspace. What happens?

- a) It works: Viewers read everything.
- b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs.
- c) It works, but returns only the first 1,000 rows.
- d) It fails because Viewers cannot see notebooks at all.

Why?

- r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).
- r2) Spark is disabled for all users in production workspaces.

Exercise 3. Architecture design (level 2)

Assign roles in the production workspace `sales_prod`.

Options: Admin, Member, Contributor, Viewer.

1. Fabric platform team (group)
2. Deployment service principal
3. Sales analysts (group)

Exercise 4. Write the function (level 3)

Write `min_role(action: str) -> str` returning the lowest workspace role that allows the action. Use these actions from the role table:

- `"connect_git"`, `"delete_workspace"`: Admin
- `"add_members"`: Member
- `"run_notebook"`, `"spark_read"`: Contributor
- `"tsql_read"`: Viewer

Raise `KeyError` for any other action.

Your code must pass these checks:

- Map each action to the lowest role in the table. Spark reads need Contributor. T-SQL reads need Viewer.
- Unknown actions must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
MIN_ROLE = ...

# Lowest Fabric workspace role that allows an action.
def min_role(action: str) -> str: ...
```

Starter code:

```
def min_role(action):
    return "Viewer"
```

Exercise 5. Spot the bug (level 4)

This role file for the production workspace breaks two rules of the lesson. Tap the line with the more dangerous one, then rewrite the file.

```
1 workspace: sales_prod
2 roles:
3   Admin: [grp-fabric-platform-admins]
4   Contributor: [sp-sales-deployment, grp-sales-analysts]
5   Viewer: [jo.bloggs@contoso.com]
```

Write down the line number of the bug. Then write the fix.

Sources

- The four workspace roles and their rights, including Admin-only Git connection and Viewer having ReadData but not ReadAll. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/roles-workspaces (updated 2026-04-24))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C1-L1 Tenant, capacity, workspace and item

C1-L1 exercise 1

Hints. (1) What does a capacity provide? (2) Compute, measured in a unit. (3) Capacity units.

Answer. b) The capacity has 32 capacity units (CUs) of compute. Because: r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.

F SKUs are named by their capacity units. F32 provides 32 CUs.

Common wrong answers.

- `a`: Capacities are not sized by user count. The number is compute in CUs.
- `c`: OneLake storage is billed separately from capacity compute.
- `d`: The SKU does not cap workspaces in this way. It measures compute.
- `b|r2`: Right answer, wrong reason. The number is exactly the CU count.

C1-L1 exercise 2

Hints. (1) Which level holds organisation-wide settings? (2) Which level provides compute to workspaces? (3) Items live in workspaces.

Answer.

1. Tenant
2. Capacity
3. Workspace
4. Item, such as a lakehouse

Tenant settings apply to all capacities. Each workspace draws compute from its capacity. Items live in workspaces.

Common wrong answers.

- A workspace is assigned to a capacity, which provides its compute. Capacity sits above workspace.
- Capacities belong to a tenant, where tenant settings apply to everything.

C1-L1 exercise 3

Hints. (1) Which SKU lets free-licence users view Power BI content? (2) What happens when two workloads share one pool of CUs? (3) Size and isolation.

Answer.

- F64 or larger, because smaller F SKUs require each viewer to hold a Pro, PPU or trial licence.
- Production only, so experiments cannot use up the compute the report needs.

Licensing and isolation both point to a dedicated F64 or larger for this report.

Common wrong answers.

- sku = `f16`: Below F64, viewers need Pro, Premium Per User or trial licences. 200 free-licence users could not open the report.
- share = `sandbox`: A heavy notebook in the sandbox consumes the same CUs and can slow or throttle the report.

C1-L1 exercise 4

Hints. (1) Build a mapping from each valid SKU name to its CUs. (2) Check membership in the mapping, then the hours. (3) Multiply and return a float.

Answer.

```python
F_SKUS: dict[str, int] = {f"F{n}": n for n in (2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048)}


def cu_hours(sku: str, hours: float) -> float:
    """Capacity-unit hours for running an F SKU capacity.

    Args:
        sku: F SKU name such as "F64".
        hours: Hours the capacity runs.

    Returns:
        CUs multiplied by hours.

    Raises:
        ValueError: For an unknown SKU or negative hours.
    """
    if sku not in F_SKUS:
        raise ValueError(f"unknown F SKU: {sku!r}")
    if hours < 0:
        raise ValueError("hours must not be negative")
    return float(F_SKUS[sku] * hours)
```

An explicit list of SKUs rejects typos such as F3 or f64 instead of pricing a capacity that does not exist.

C1-L1 exercise 5

Hints. (1) Was the afternoon's compute already used? (2) What happens to usage Fabric smoothed over the coming hours? (3) It is settled at pause.

Answer. b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes. Because: r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.

Pausing saves money from then on, but the usage already smoothed forward is billed at once, and every item on the capacity stops until it resumes.

Common wrong answers.

- `a`: The work already happened. Pausing settles the remaining smoothed and overage usage on the bill.
- `c`: A paused capacity is not operational. Its items stop until you resume it.
- `d`: Pausing affects compute. It does not delete OneLake data.
- `b|r2`: Right answer, wrong reason. There is no double rate. Pausing settles usage already incurred.

C1-L2 OneLake, one lake per tenant

C1-L2 exercise 1

Hints. (1) Can you create a second OneLake? (2) What is a workspace in OneLake terms? (3) A folder.

Answer. b) One for the whole tenant Because: r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.

One tenant, one OneLake. Workspaces and items are folders inside it.

Common wrong answers.

- `a`: You cannot create more OneLakes. Business units get workspaces and domains inside the one lake.
- `c`: Capacities provide compute. Storage lives in the tenant's single OneLake.
- `d`: Each workspace is a folder in OneLake, not a separate lake.
- `b|r2`: Right answer, wrong reason. It is a platform design, not a licensing unit.

C1-L2 exercise 2

Hints. (1) The workspace goes before the @. (2) The item name carries its type in lower case. (3) Tables for managed Delta tables.

Answer.

```
abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/Tables/orders
```

Workspace before `@`, then `item.itemtype`, then `Tables/<table>`.

Common wrong answers.

- g3 = `Files`: Files holds unmanaged files. Managed Delta tables live under Tables.
- g1 = `lh_sales`: The part before @ is the workspace, not the item.

C1-L2 exercise 3

Hints. (1) What character does the workspace name contain? (2) The abfs URI does not allow it. (3) Use the GUIDs, which never change.

Answer. The bug is on line 1.

Fixed version:

```
WORKSPACE_ID = CONFIG["workspace_id"]
LAKEHOUSE_ID = CONFIG["lakehouse_id"]
path = f"abfss://{WORKSPACE_ID}@onelake.dfs.fabric.microsoft.com/{LAKEHOUSE_ID}/Tables/orders"
```

The space in the workspace name breaks the abfs URI. Renaming the workspace would fix this job and break everything else that uses the old name. GUIDs fix both.

Common wrong answers.

- Line 3: The format of line 3 is right for names. The value it receives on line 1 is the problem.

C1-L2 exercise 4

Hints. (1) Check the item type against a fixed set. (2) Reject spaces in workspace and item names. (3) relative.lstrip('/') avoids a double slash.

Answer.

```
ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
ITEM_TYPES = frozenset({"lakehouse", "warehouse"})


def onelake_path(workspace: str, item: str, item_type: str, relative: str) -> str:
    """abfss path to data in a OneLake item, addressed by name.
```

```
7
8       Args:
9           workspace: Workspace name, without spaces.
10          item: Item name, without spaces.
11          item_type: "lakehouse" or "warehouse".
12          relative: Path inside the item, such as Tables/orders.
13
14      Returns:
15          The abfss URI.
16
17      Raises:
18          ValueError: For an unknown item type or names with spaces.
19      """
20      if item_type not in ITEM_TYPES:
21          raise ValueError(f"unsupported item type: {item_type!r}")
22      if " " in workspace or " " in item:
23          raise ValueError("abfs URIs do not allow spaces; use GUIDs
    instead")
24      return f"abfss://{workspace}@{ONELAKE_HOST}/{item}.{item_type}/
    {relative.lstrip('/')}"
```

Validating inputs turns a confusing URI error at read time into a clear message at build time.

C1-L2 exercise 5

Hints. (1) What changes and what does not? (2) Which endpoint guarantees the region? (3) Does Databricks need a copy to read Delta?

Answer.

- GUIDs survive renames and avoid URI problems with special characters.
- The regional endpoint keeps requests in the workspace's region, which the residency rule needs.
- OneLake speaks ADLS APIs and stores open Delta tables, so the job reads them without a second copy.

Stable identifiers, a regional endpoint, and no copies.

Common wrong answers.

- address = `names`: Names change, and names with spaces break abfs URIs.
- endpoint = `global`: Microsoft Learn warns that the global endpoint can route data out of your region.
- copy = `copy_nightly`: A nightly copy is a second dataset to govern and keep in sync. Read the Delta tables where they are.

C1-L3 Workspace roles, granted to groups

C1-L3 exercise 1

Hints. (1) Which rights sit only at the top of the role table? (2) Workspace-level settings. (3) Admin.

Answer. a) Admin only Because: r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.

Only Admins connect a workspace to Git, create a workspace identity, and update or delete the workspace.

Common wrong answers.

- `b`: Members can add people and reshare, but connecting Git is an Admin right.
- `c`: Contributors create and change items, but cannot connect the workspace to Git.
- `d`: Viewers cannot change anything in the workspace.
- `a|r2`: Right answer, wrong reason. It is a permission boundary, not a licence.

C1-L3 exercise 2

Hints. (1) Which two read rights does the role table list? (2) Which one does Viewer have? (3) ReadData, the T-SQL one.

Answer. b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs. Because: r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).

Point Viewers to the SQL analytics endpoint, or grant ReadAll on the item if Spark access is really needed.

Common wrong answers.

- `a`: Viewer grants T-SQL reads only. Spark reads need ReadAll, which starts at Contributor.
- `c`: There is no row limit tied to the role. The read is refused.
- `d`: Viewers can open and read notebooks. They cannot run them or read data through Spark.
- `b|r2`: Right answer, wrong reason. Spark is available. The Viewer role lacks ReadAll.

C1-L3 exercise 3

Hints. (1) Who manages, who writes, who reads? (2) Least privilege. (3) Groups, not individuals.

Answer.

- Platform team: Admin, as a group, so management does not depend on one person.
- Deployment identity: Contributor, the only writer in production.
- Analysts: Viewer. They query with T-SQL and use reports.

One managing group, one automated writer, and readers as Viewers.

Common wrong answers.

- analysts = `contributor`: Contributors can change and delete every item in production. Analysts need to read.
- deploy = `admin`: Deployment needs to write items, not manage people or delete the workspace. Contributor is enough.
- platform = `viewer`: Someone must be able to manage the workspace. Give that right to a group.

C1-L3 exercise 4

Hints. (1) A dictionary from action to role. (2) Spark and OneLake reads (ReadAll) start at Contributor. (3) Indexing a dict raises KeyError for unknown keys.

Answer.

```python
MIN_ROLE: dict[str, str] = {
    "connect_git": "Admin",
    "delete_workspace": "Admin",
    "add_members": "Member",
    "run_notebook": "Contributor",
    "spark_read": "Contributor",
    "tsql_read": "Viewer",
}


def min_role(action: str) -> str:
    """Lowest Fabric workspace role that allows an action.

    Raises:
        KeyError: For an action not in the table.
    """
    return MIN_ROLE[action]
```

Encoding the role table as data lets access requests be checked automatically, and an unknown action fails loudly instead of defaulting to a guess.

C1-L3 exercise 5

Hints. (1) Which line grants write access? (2) Do analysts need to change production items? (3) Also replace the individual with a group.

Answer. The bug is on line 4.

Fixed version:

```yaml
workspace: sales_prod
roles:
  Admin: [grp-fabric-platform-admins]
  Contributor: [sp-sales-deployment]
  Viewer: [grp-sales-analysts, grp-sales-readers]
```

Analysts as Contributors can change or delete production items. Make them Viewers, keep automation as the only writer, and replace the individual with a group.

Common wrong answers.

- Line 5: An individual assignment breaks the groups rule, but it only grants read access. Line 4 gives write access to production.
- Line 3: An admin group is the right pattern.